

Final Examination

Directions:

1. Submit the complete Chapters 1-3 manuscript with a link to the datasets included at the end of the document.
2. Submit a 7-10 minute pre-recorded video presenting your research collaboration progress via the provided Google Sheet link.
3. Follow the specifications below for your video presentation.
4. Practice professional presentation etiquette: dress appropriately, speak clearly, minimize filler words, and respect your audience by adhering to the time limit.
5. Due on May 23, 2026.

A. Video Specifications

Duration: 7-10 minutes maximum (5-7 minutes presentation + 2-3 minutes for demo/technical walkthrough)

Format: Pre-recorded video combining slides and live system demonstration

Technical Requirements:

- Minimum resolution: 1080p (HD) - **1440p recommended for game/simulation visuals**
- Screen recording with clear audio narration
- **60 FPS recording preferred for game demos** to showcase smooth gameplay
- Include code snippets, system demonstrations, or algorithm visualizations
- File format: .mp4 or .mov
- File size: Maximum 1GB (compress if needed while maintaining visual quality)
- File naming: LastName_FirstName_RC_CS_May2026

Submission Deadline: May 17, 2026, 11:59 PM (allows technical review before presentation date)

B. Content Structure for CS Research

1. Title Slide (30 seconds)

- Project/Research title
- Student name, student ID, and program (BS/MS CS)
- **Track/Specialization:** (General CS / Game Development / Simulation Development)
- Research advisor/supervisor name
- Date: May 18, 2026
- GitHub repository, itch.io, or project URL (if applicable)
- **Game/Simulation Genre or Application Domain** (if applicable)

2. Problem Statement & Motivation (1 minute)

For General CS Projects:

- What specific CS problem are you solving?
- Real-world application or use case
- Why is this technically challenging or interesting?
- Related to: AI/ML, Data Mining, NLP, Software Engineering, HCI, Assistive Technology, etc.
- Connection to SDGs or societal impact (especially for eGovernment, eLearning applications)

For Game Development Projects:

- What gaming experience or problem are you creating/solving?
- Target audience and platform (PC, mobile, web, VR/AR)
- Game genre and core gameplay mechanics
- Educational, entertainment, or serious game purpose
- How does it contribute to game design knowledge or accessibility?

For Simulation Development Projects:

- What real-world system or phenomenon are you simulating?
- Application domain (physics, biology, social systems, training, urban planning, etc.)
- Why simulation is necessary (cost, safety, complexity, research)
- Expected users and use cases
- Research or practical problem it addresses

3. Research Objectives & Scope (45 seconds)

- 2-4 specific, measurable objectives
- Technical contributions expected
- Scope boundaries (what's included/excluded)
- Success criteria or evaluation metrics
- **For games:** Player experience goals, learning outcomes, or engagement metrics
- **For simulations:** Accuracy targets, validation criteria, or performance benchmarks

4. Literature Review & Related Work (1 minute)

- Key papers, games, or systems you're building upon
- Comparison with existing solutions (create comparison table if applicable)
- **For games:** Similar games analyzed (mechanics, art style, technology)

- **For simulations:** Existing simulation models or frameworks reviewed
- Research gap you're addressing
- Your proposed approach vs. state-of-the-art

5. Methodology & System Architecture (1.5-2 minutes)

For General Software Development Projects:

- System architecture diagram
- Technology stack (frameworks, libraries, APIs, databases)
- Development methodology (Agile, Scrum, iterative)
- Tools used (IDEs, version control, testing frameworks)

For AI/ML/Data Mining Projects:

- Dataset description (size, source, preprocessing steps)
- Algorithm or model selection rationale
- Training/testing split and validation approach
- Evaluation metrics (accuracy, precision, recall, F1-score, etc.)

For Game Development Projects - Include:

- **Game engine:** Unity, Unreal Engine, Godot, custom engine, or framework
- **Core systems architecture:** Game loop, entity-component system, state machines
- **Technology stack:**
 - Programming languages (C#, C++, Python, JavaScript)
 - Graphics API (DirectX, OpenGL, Vulkan, WebGL)
 - Physics engine (built-in or third-party)
 - Audio system
 - Asset pipeline
- **Game design document highlights:** Core mechanics, player goals, progression system
- **Art style and visual direction:** 2D/3D, pixel art, low-poly, realistic, stylized
- **Development tools:** Blender, Maya, Photoshop, Aseprite, ProBuilder
- **Level design approach:** Procedural generation, hand-crafted, hybrid

For Simulation Development Projects - Include:

- **Simulation paradigm:** Discrete-event, continuous, agent-based, Monte Carlo, hybrid
- **Mathematical/computational model:** Equations, algorithms, or rules governing simulation

- **Simulation engine/framework:** Unity (for 3D), NetLogo, AnyLogic, MATLAB/Simulink, custom
- **Technology stack:**
 - Programming languages and libraries
 - Numerical methods or solvers used
 - Visualization tools (matplotlib, Plotly, Unity, Unreal)
 - Data collection and analysis tools
- **Model parameters and variables:** Key inputs, state variables, outputs
- **Validation strategy:** How you ensure simulation accuracy (comparison with real data, expert validation)
- **Scalability considerations:** Number of agents/entities, time steps, spatial resolution

For Algorithm Research:

- Pseudocode or flowcharts
- Complexity analysis approach
- Test case design

6. Progress Update & Implementation Status (2.5-3 minutes)



- Code statistics (lines of code, files, commits)
- **For games:** Levels/scenes completed, mechanics implemented, assets created
- **For simulations:** Model components completed, validation tests run

Technical Demonstrations:

A. For General CS Projects:

- Live system demo or screen recording (30-60 seconds)
- Show 2-4 key features working
- Demonstrate user interface or command-line operation
- Show input → processing → output flow

B. For Game Development Projects - Show:

- **Gameplay footage (60-90 seconds):**
 - Opening scene or main menu
 - Core gameplay loop demonstration
 - 2-3 key mechanics in action
 - Player interactions and controls
 - UI/UX elements
 - Visual effects and animations
 - Audio/sound effects (if implemented)
- **Technical features:**
 - Camera system, character controller
 - AI behavior (enemy AI, NPC pathfinding)
 - Physics interactions
 - Procedural generation (if applicable)
 - Save/load system
 - Multiplayer functionality (if applicable)
- **Asset showcase:** Character models, environments, animations created
- **Behind-the-scenes:** Level editor, tool development, or debugging process

C. For Simulation Development Projects - Show:

- **Simulation run (60-90 seconds):**
 - Initial conditions and parameter settings
 - Simulation execution in real-time or time-lapse
 - Visualization of system behavior over time
 - Dynamic changes and interactions
 - Multiple scenarios or parameter variations
- **Technical features:**
 - Agent behaviors or entity interactions
 - Boundary conditions and constraints
 - Real-time data collection
 - Statistical outputs or metrics
 - 3D visualization or graphs
 - Interactive controls (pause, speed up, reset)

- **Validation results:** Comparison with expected behavior, real-world data, or theoretical predictions
- **Performance metrics:** Frames per second, simulation speed, scalability tests

Code Snippets (All Tracks):

- Highlight 1-2 critical code segments
- **For games:** Show player controller, game manager, AI behavior script, or shader code
- **For simulations:** Show core simulation loop, agent update logic, or mathematical model implementation
- Explain key algorithms or functions
- Show coding best practices applied

Preliminary Results:

- Performance metrics, benchmarks, or test results
- **For games:** Playtesting feedback, player engagement data, performance FPS
- **For simulations:** Accuracy measurements, convergence tests, sensitivity analysis
- Graphs/charts showing system performance
- Comparison with baseline or expected results

Challenges & Solutions:

- Technical obstacles encountered
- **For games:** Physics bugs, performance optimization, balancing issues, art pipeline problems
- **For simulations:** Numerical stability, computational complexity, validation difficulties
- How you debugged or resolved them
- Lessons learned



7. Experimental Results & Analysis (1 minute)

For General CS Projects:

- Data visualizations (confusion matrices, ROC curves, bar charts)
- Statistical analysis results
- Performance comparisons across different configurations

For Game Development Projects:

- **Playtesting results:** Number of testers, feedback summary, usability findings
- **Performance analysis:** Frame rates across different hardware, loading times, memory usage
- **Player metrics:** Completion rates, time spent, difficulty progression
- **Bug tracking:** Issues found and resolved
- **Educational games:** Learning outcomes assessment, pre/post-test results

For Simulation Development Projects:

- **Validation results:** Comparison with real-world data or analytical solutions
- **Sensitivity analysis:** How outputs vary with parameter changes
- **Statistical outputs:** Mean, variance, confidence intervals from multiple runs
- **Scalability tests:** Performance with varying numbers of agents/entities
- **Visual analysis:** Heatmaps, trajectory plots, time-series graphs
- **Model verification:** Unit tests for individual components

8. Remaining Work & Timeline (1 minute)

For All Tracks:

- Outstanding tasks with specific completion dates
- Testing and validation plans
- Documentation requirements
- Deployment or integration steps
- Risk mitigation for potential blockers

Additional for Game Development:

- Art assets to complete
- Levels/content to create
- Audio implementation
- Polish and juice (particle effects, screen shake, feedback)
- Final playtesting and balancing
- Build optimization for target platforms

Additional for Simulation Development:

- Additional validation scenarios
- Parameter optimization
- User interface improvements
- Documentation of model assumptions
- Sensitivity analysis completion
- Publication or deployment preparation

9. Expected Contributions & Impact (30 seconds)

- Key technical insights or innovations
- Academic contributions (publications planned)
- Practical applications
- **For games:** Contribution to game design knowledge, accessibility, education, or cultural impact
- **For simulations:** Scientific insights, training applications, policy implications, research tool development
- Future research directions
- How this advances the field

10. References & Acknowledgments (15 seconds)

- Key references cited (3-5 major papers/sources/games/simulations)
- Acknowledgment of advisor, collaborators, funding
- Asset credits (if using third-party assets with proper licensing)

C. Technical Presentation Guidelines

Code & Algorithm Presentation:

- **Syntax highlighting:** Use proper code formatting
- **Font size:** Minimum 14pt for code blocks
- **Keep it concise:** Show only essential code segments (10-15 lines max per slide)
- **Explain logic:** Narrate what the code does, don't just read it
- **Use animations:** Show algorithm execution step-by-step for complex processes

System Demonstration Best Practices:

- **Script your demo:** Plan exactly what to show (5-13 steps recommended)
- **Highlight key features:** 2-4 "aha moments" where functionality impresses
- **Use visual indicators:** Arrows, circles, or highlights to guide viewer attention

- **Test beforehand:** Record demo separately to ensure smooth playback
- **Narrate clearly:** Explain what you're clicking and why
- **Show error handling:** Demonstrate robustness (if applicable)

Game Demo Best Practices:

- **Show, don't tell:** Let gameplay speak for mechanics
- **Smooth recording:** Use 60 FPS capture for fluid motion
- **Audio balance:** Game audio should complement, not overpower narration
- **Controlled environment:** Demo in optimal conditions (good lighting, no distractions)
- **Highlight innovation:** Focus on unique mechanics or features
- **Show polish:** Demonstrate finished sections rather than everything partially done
- **Player perspective:** Show from player's viewpoint, explain controls
- **Multiple scenarios:** Show variety (different levels, characters, situations)

Simulation Demo Best Practices:

- **Clear visualization:** Use color coding, labels, legends effectively
- **Multiple views:** Show different perspectives (overview, detailed, graphs)
- **Explain what's happening:** Narrate simulation behavior and why it matters
- **Show parameter effects:** Demonstrate how changing inputs affects outputs
- **Comparison:** Show baseline vs. optimized or different scenarios side-by-side
- **Speed control:** Time-lapse long simulations, slow-mo for critical moments
- **Data interpretation:** Connect visual simulation to quantitative results
- **Validation evidence:** Show how simulation matches expected/real behavior

Visual Design for CS Presentations:

- Use professional presentation software (PowerPoint, Google Slides, LaTeX Beamer)
- Maintain consistent theme and color scheme
- **Architecture diagrams:** Use tools like draw.io, Lucidchart, or PlantUML
- **Flowcharts:** Show algorithm logic flow clearly
- **Screenshots:** Annotate with callouts to highlight important elements
- **Game/Simulation visuals:** High-quality captures, proper aspect ratio
- **Limit text:** Use bullet points, not paragraphs (max 5-6 bullets per slide)
- **Data visualization:** Use appropriate chart types (line for trends, bar for comparisons)

Recording Tools & Software

Screen Recording:

- OBS Studio (free, open-source, excellent for game capture)
- NVIDIA ShadowPlay (for NVIDIA GPU users, minimal performance impact)

- Xbox Game Bar (Windows built-in)
- Zoom (record presentation mode)
- Loom or Camtasia (easy editing)

Game/Simulation Capture:

- Unity Recorder (built-in Unity asset)
- Unreal Engine Sequencer
- OBS Studio with game capture source
- Fraps or Bandicam (dedicated game recording)

Code Demonstration:

- Use IDE with readable theme (consider light themes for better visibility)
- Zoom in or increase font size (minimum 18pt)
- Carbon.now.sh for beautiful code screenshots

Video Editing (if needed):

- DaVinci Resolve (free, professional)
- Adobe Premiere Pro
- Camtasia
- OpenShot (free, simple)

Submission Requirements Checklist

All Students:

- Video length: 7-10 minutes total
- Includes working system demonstration or code walkthrough
- All technical diagrams are clear and readable
- Code snippets are properly formatted with syntax highlighting
- Audio is synchronized and free from background noise
- Results/data visualizations are included
- File size is manageable (maximum 1GB)
- GitHub/repository link provided (if applicable)
- References properly cited
- Submitted in correct format with proper filename

Additional for Game Development:

- Gameplay footage is smooth (minimum 30 FPS in recording)
- Game controls/mechanics are clearly explained
- Audio levels are balanced (narration audible over game audio)
- Build runs without critical bugs during demo
- Credits for third-party assets included
- itch.io or playable demo link (if available)

Additional for Simulation Development:

- Simulation visualization is clear and labeled
- Parameters and initial conditions are stated
- Validation approach is explained
- Statistical or quantitative results included
- Multiple simulation scenarios shown (if applicable)
- Model assumptions are documented

Tips

General CS Students

1. **Balance theory and practice:** Don't just explain algorithms theoretically—show them working
2. **Use analogies:** Explain complex CS concepts with real-world analogies
3. **Avoid jargon overload:** Define technical terms for broader audience
4. **Show, don't just tell:** Live demos are more convincing than static descriptions
5. **Version control matters:** Mention Git commits, branches, version history

Game Development Students

1. **Lead with gameplay:** Show your game in action early to engage viewers
2. **Explain player experience:** Describe what players feel, not just what code does
3. **Show iteration:** Briefly mention how playtesting improved your design
4. **Balance fun and tech:** Don't just showcase technology—show why it's enjoyable
5. **Credit your assets:** Be transparent about what you created vs. what you used
6. **Performance matters:** Discuss optimization efforts and frame rate targets
7. **Accessibility:** Mention any inclusive design features (subtitles, colorblind modes, difficulty options)
8. **Know your audience:** Tailor complexity to evaluators' game development knowledge

Simulation Development Students

1. **Justify your model:** Explain why you chose specific approaches or equations
2. **Show validation rigorously:** Demonstrate that simulation matches reality or theory
3. **Visualize effectively:** Good visualization makes complex behavior understandable
4. **Discuss limitations:** Be honest about assumptions and simplifications
5. **Show parameter sensitivity:** Demonstrate you understand your model's behavior
6. **Connect to application:** Always link technical details to real-world use case
7. **Statistical rigor:** Include confidence intervals, error bars, multiple runs
8. **Computational efficiency:** Discuss performance and scalability considerations

Common Mistakes to Avoid

All Tracks:

- Spending too much time on background/related work (max 1 minute)
- Showing code without explaining its purpose
- Poor audio quality during demonstrations
- Text too small to read on diagrams
- Rushing through the most important technical contributions
- Ignoring negative or unexpected results

Game Development Specific:

- Demo footage with obvious bugs or crashes
- Showing too many unfinished features instead of polished core mechanics
- Game audio drowning out narration
- Not explaining controls or what player should do
- Only showing static screenshots instead of gameplay
- Overlong gameplay without commentary

Simulation Development Specific:

- Running simulation without explaining what's being modeled
- No validation or accuracy discussion
- Showing simulation without quantitative results
- Unclear visualization (missing labels, legends, units)
- Not explaining parameter choices
- Ignoring edge cases or unexpected behavior
- No comparison with baseline or alternative approaches

Evaluation Rubric

Criteria	Excellent (90-100%)	Proficient (75-89%)	Developing (60-74%)	Needs Improvement (<60%)	Points
1. Introduction & Problem Statement	Clear, compelling problem with strong motivation. Excellent connection to real-world applications and societal impact.	Clear problem statement with good motivation. Adequate connection to applications.	Problem stated but lacks clarity or motivation. Weak connection to applications.	Unclear or missing problem statement. No motivation provided.	/10

2. Research Objectives & Methodology	2-4 specific, measurable objectives. Comprehensive methodology with clear architecture, technology stack, and validation approach.	Clear objectives with good methodology. Most technical details provided.	Objectives present but vague. Methodology lacks some important details.	Unclear objectives. Methodology missing or inadequate.	/15
3. Implementation Quality & Progress	Excellent progress (>70% complete). All milestones clearly documented. High-quality code and assets demonstrated.	Good progress (50-70%). Most milestones achieved. Quality implementation shown.	Moderate progress (30-50%). Some milestones missing. Implementation needs work.	Minimal progress (<30%). Few milestones achieved. Poor implementation quality.	/20
4. Technical Demonstration	Outstanding live demo (60-90 sec). Shows 3-4 key features working flawlessly. Smooth, professional, and engaging. Well-narrated.	Good demo showing 2-3 features. Minor issues but overall effective. Clear narration.	Basic demo with 1-2 features. Some technical problems. Narration needs improvement.	Poor or missing demo. Major technical issues. Unclear what is being shown.	/25
5. Code Quality & Documentation	Excellent code snippets (1-2) with proper syntax highlighting. Clear explanation of algorithms. Best practices evident.	Good code presentation. Adequate explanations. Some best practices shown.	Code shown but poorly formatted or explained. Few best practices.	Missing or illegible code. No explanation of logic.	/10
6. Results & Analysis	Comprehensive results with multiple visualizations. Statistical analysis included. Clear interpretation with comparisons to baseline/expectations.	Good results with visualizations. Basic analysis provided. Some comparisons made.	Limited results or visualizations. Minimal analysis. Few comparisons.	Missing or inadequate results. No analysis or visualization.	/10
7. Technical Quality	Professional video (1080p+, 60fps for games). Crystal clear audio. Excellent slide design. All text readable (14pt+). Smooth transitions.	Good video quality (1080p). Clear audio. Good slides. Most text readable.	Adequate video quality. Some audio issues. Slides need improvement. Text sometimes hard to read.	Poor video/audio quality. Unprofessional slides. Text illegible.	/10
Total					/100

